

Roman Numerals to Integer

Problem Statement

Given a string `s` representing a Roman numeral, convert it into its corresponding integer value.

Roman numeral symbols and their values are:

Symbol	Value
I	1
V	5
X	10
L	50
C	100
D	500
M	1000

Roman numerals normally follow descending order, but there are special cases where a smaller value appears before a larger value.

Examples:

```
IV = 4
IX = 9
XL = 40
XC = 90
CD = 400
CM = 900
```

Example 1

Input

```
s = "III"
```

Output

3

Explanation

```
I + I + I  
= 1 + 1 + 1  
= 3
```

Example 2

Input

s = "LVIII"

Output

58

Explanation

```
L + V + I + I + I  
= 50 + 5 + 1 + 1 + 1  
= 58
```

Example 3

Input

s = "MCMIV"

Output

1904

Explanation

```
M    CM    IV
1000 + 900 + 4
= 1904
```

Approach

The solution uses:

- `HashMap<Character, Integer>` to store Roman numeral values.
- `char[]` to access each character.
- Right-to-left traversal.
- Subtraction when a smaller Roman value appears before a larger value.

The main idea is:

```
If current value > previous value:
    subtract previous value
Otherwise:
    add current value
```

Your implementation handles the subtraction pair together.

Java Implementation

```
class Solution {
    public int romanToInteger(String s) {

        HashMap<Character, Integer> map = new HashMap<>();

        map.put('I', 1);
        map.put('V', 5);
        map.put('X', 10);
        map.put('L', 50);
```

```

        map.put('C', 100);
        map.put('D', 500);
        map.put('M', 1000);

        char[] arr = s.toCharArray();

        int count = 0;

        for (int i = arr.length - 1; i >= 0; i--) {

            if (i > 0 && map.get(arr[i]) > map.get(arr[i - 1])) {

                count += map.get(arr[i]) - map.get(arr[i - 1]);

                i--;
            }
            else {
                count += map.get(arr[i]);
            }
        }

        return count;
    }
}

```

Code Explanation

1. Create HashMap

```
HashMap<Character, Integer> map = new HashMap<>();
```

The map stores each Roman symbol and its corresponding value.

For example:

```

I → 1
V → 5
X → 10
L → 50
C → 100

```

```
D → 500  
M → 1000
```

This allows us to get the value of a Roman character using:

```
map.get(character)
```

2. Convert String to Character Array

```
char[] arr = s.toCharArray();
```

For:

```
s = "MCMIV"
```

we get:

```
arr = ['M', 'C', 'M', 'I', 'V']
```

Indexes:

```
      0    1    2    3    4  
-----  
arr = M    C    M    I    V
```

3. Initialize Result

```
int count = 0;
```

`count` stores the final integer value.

Initially:

```
count = 0
```

4. Traverse From Right to Left

```
for (int i = arr.length - 1; i >= 0; i--)
```

Instead of moving:

```
left → right
```

we move:

```
right → left
```

For:

```
M C M I V
```

the traversal is:

```
V → I → M → C → M
```

This makes it easy to identify subtraction cases.

5. Check the Subtraction Condition

The important condition is:

```
if (i > 0 && map.get(arr[i]) > map.get(arr[i - 1]))
```

There are two conditions.

Condition 1

```
i > 0
```

This makes sure that `arr[i - 1]` exists.

If:

```
i = 0
```

then:

```
arr[i - 1] = arr[-1]
```

would be invalid.

Condition 2

```
map.get(arr[i]) > map.get(arr[i - 1])
```

This checks whether the current Roman value is greater than the value immediately before it.

For:

```
IV
```

we traverse from right to left:

```
V → I
```

At `V`:

```
V = 5  
I = 1
```

Since:

```
5 > 1
```

we know:

```
IV = 5 - 1 = 4
```

6. Add the Subtraction Pair

```
count += map.get(arr[i]) - map.get(arr[i - 1]);
```

For:

```
IV
```

this becomes:

```
count += 5 - 1
```

Therefore:

```
count = 4
```

7. Why `i--` Is Used

This is an important part of your code:

```
i--;
```

Suppose:

```
s = "IV"
```

Initially:


```
i = 1
```

We process:

```
V
```

and detect that:

```
V > I
```

So we process the entire pair:

```
IV
```

using:

```
count += 5 - 1;
```

Now **I** has already been processed.

Therefore:

```
i--;
```

changes:

```
i = 1
```

to:

```
i = 0
```

Then the **for** loop itself performs another decrement:

```
i--
```

so:

```
i = -1
```

The loop terminates.

In simple words

`i--` is used because **both characters of the subtraction pair have already been processed.**

```
IV
↑↑
processed together
```

Without `i--`, the `I` would be processed again and incorrectly added.

8. Normal Addition Case

If the subtraction condition is false:

```
else {
    count += map.get(arr[i]);
}
```

The current value is simply added.

For example:

```
III
```

Every `I` is added:

```
1 + 1 + 1 = 3
```

Dry Run 1: "III"

Input:

```
s = "III"
```

Array:

```
I I I  
0 1 2
```

Iteration 1

```
i = 2  
current = I = 1  
previous = I = 1
```

Check:

```
1 > 1 → false
```

So:

```
count = 0 + 1 = 1
```

Iteration 2

```
i = 1
```

Again:

```
1 > 1 → false
```

```
count = 1 + 1 = 2
```

Iteration 3

```
i = 0
```

`i > 0` is false.

So:

```
count = 2 + 1 = 3
```

Final:

```
3
```

Dry Run 2: "IV"

Input:

```
IV
```

Array:

```
I V  
0 1
```

Iteration 1

```
i = 1
```

Current:

```
V = 5
```

Previous:

```
I = 1
```

Check:

```
5 > 1
```

True.

Therefore:

```
count += 5 - 1  
count = 4
```

Then:

```
i--;
```

The loop terminates.

Final:

```
4
```

Dry Run 3: "MCMIV"

Input:

```
MCMIV
```

Array:

```
M C M I V  
0 1 2 3 4
```

We start from the right.

i	Current	Previous	Action	count
4	V = 5	I = 1	5 - 1	4
2	M = 1000	C = 100	1000 - 100	904
0	M = 1000	—	+1000	1904

Final answer:

1904

Notice that after processing:

IV

we skip both characters.

Likewise:

CM

is processed together.

Important Subtraction Cases

The six standard subtraction combinations are:

IV = 4
IX = 9

XL = 40
XC = 90

CD = 400
CM = 900

The logic:

```
map.get(arr[i]) > map.get(arr[i - 1])
```

detects these patterns when traversing from right to left.

Why Right-to-Left?

Consider:

MCM

If we move left to right, we need to look ahead to determine whether a smaller value should be subtracted.

But when moving right to left:

M C M
 ↑

we already know the larger value on the right.

For:

CM

we see:

M = 1000
C = 100

Since:

1000 > 100

we calculate:

1000 - 100 = 900

This makes the subtraction logic straightforward.

Algorithm

1. Create a HashMap containing Roman symbols and their values.
2. Convert the input string into a character array.
3. Initialize count = 0.
4. Traverse the array from right to left.
5. If current value > previous value:
 Add current - previous to count.
 Decrement i to skip the already processed pair.
6. Otherwise:
 Add current value to count.
7. Return count.

Edge Cases

Single Symbol

Input:
"X"

Output:
10

Normal Roman Numeral

Input:
"VIII"

Output:
8

Subtraction

Input:
"IX"

Output:
9

Multiple Subtractions

Input:
"MCMIV"

Output:
1904

Complexity Analysis

Let n be the length of the Roman numeral.

Time Complexity

We traverse the string once:

$O(n)$

HashMap lookup is $O(1)$ on average.

Therefore:

Time Complexity = $O(n)$

Space Complexity

The HashMap contains only 7 Roman symbols:

I, V, X, L, C, D, M

The character array requires $O(n)$ space.

Therefore, with your exact implementation:

Space Complexity = $O(n)$

The HashMap itself is constant space:

$O(1)$

Key DSA Concepts

This problem demonstrates:

- HashMap
- Character array
- String manipulation
- Reverse traversal
- Conditional subtraction
- `StringBuilder` is not required here
- Greedy-style local comparison

Key Takeaway

The most important idea is:

Traverse from RIGHT → LEFT

Then:

```
if current > previous
    ↓
    subtract previous from current

otherwise
    ↓
    add current
```

Your implementation additionally processes subtraction pairs together:

```
count += map.get(arr[i]) - map.get(arr[i - 1]);  
i--;
```

The `i--` is essential because the previous character has already been included in the subtraction.

Final Complexity

```
Time Complexity : O(n)  
Space Complexity : O(n)
```

Core pattern:

```
Roman String  
  ↓  
HashMap Values  
  ↓  
Right-to-Left Traversal  
  ↓  
Compare Current & Previous  
  ↓  
Subtract or Add  
  ↓  
Integer Result
```